

When to Stop? Towards Efficient Code Generation in LLMs with Excess Token Prevention

Lianghong Guo

Sun Yat-sen University
Zhuhai, China
guolh8@mail2.sysu.edu.cn

Yanlin Wang*

Sun Yat-sen University
Zhuhai, China
wangylin36@mail.sysu.edu.cn

Ensheng Shi

Xi'an Jiaotong University
Xi'an, China
s1530129650@stu.xjtu.edu.cn

Wanjun Zhong

Sun Yat-sen University
Guangzhou, China
zhongwj25@mail2.sysu.edu.cn

Hongyu Zhang

Chongqing University
Chongqing, China
hyzhang@cqu.edu.cn

Jiachi Chen

Sun Yat-sen University
Zhuhai, China
chenjch86@mail.sysu.edu.cn

Ruikai Zhang

Huawei Cloud Computing
Technologies Co., Ltd.
Shenzhen, China
zhangruikai1@huawei.com

Yuchi Ma

Huawei Cloud Computing
Technologies Co., Ltd.
Shenzhen, China
mayuchi1@huawei.com

Zibin Zheng

Sun Yat-sen University
Zhuhai, China
zhzibin@mail.sysu.edu.cn

Abstract

Code generation aims to automatically generate code snippets that meet given natural language requirements and plays an important role in software development. Although Code LLMs have shown excellent performance in this domain, their long generation time poses a signification limitation in practice use. In this paper, we first conduct an in-depth preliminary study with different Code LLMs on code generation tasks and identify a significant efficiency issue, i.e., *continual generation of excess tokens*. It harms the developer productivity and leads to huge computational wastes. To address it, we introduce **CODEFAST**, an inference acceleration approach for Code LLMs on code generation. The key idea of CODEFAST is to terminate the inference process in time when unnecessary excess tokens are detected. First, we propose an automatic data construction framework to obtain training data. Then, we train a unified lightweight model GenGuard applicable to multiple programming languages to predict whether to terminate inference at the current step. Finally, we enhance Code LLM with GenGuard to accelerate its inference in code generation tasks. We conduct extensive experiments with CODEFAST on five representative Code LLMs across four widely used code generation datasets. Experimental results show that (1) CODEFAST can significantly improve the inference speed of various Code LLMs in code generation, ranging from 34% to 452%, without compromising the quality of generated code. (2) CODEFAST is stable across different parameter settings and can generalize to untrained datasets. Our code and data are available at <https://github.com/DeepSoftwareAnalytics/CodeFast>.

1 Introduction

Code generation aims to automatically generate code snippets that meet given natural language requirements and plays an important role in software development [1, 5, 6, 18–20, 22, 25–29, 31, 33–38, 42–46, 48, 52, 56–64, 67, 68]. Recently, an increasing number of large

Prompt	Method signature (Line 1)	1	def max_of_nth(test_list, N):
		2	'''
	Method description (Lines 3-5)	3	Write a function to find the maximum of nth
		4	column from the given tuple list. '''
		5	assert max_of_nth([(5,6,7),(1,3,5),(8,9,10)],2)==19'''
Output	Expected generation (Line 6)	6	# Your code here
		7	return max(test_list,key=lambda x: x[N])
	Excess tokens (Lines 7-12)	8	def max_of_nth_2(test_list, N):
		9	'''
		10	Write a function to find the maximum of nth
		11	column from the given tuple list.
		12	assert max_of_nth_2([(5,6,7),(1,3,5),(8,9,10)],2)==19'''
			# Your code here
			return max(test_list,key=lambda x: x[N])

Figure 1: A motivating example of code generated from CodeLlama-7B.

language model for code (Code LLMs), such as Codex [38], StarCoder [28], and Code Llama [44], have achieved remarkable performance in code generation [64, 67]. Among them, Code Llama [44] stands out as a powerful open-source model in code generation tasks. In particular, models fine-tuned based on Code Llama, such as WizardCoder [33] and Phind CodeLlama [42], have demonstrated performance surpassing GPT-4 [41] in code generation.

While Code LLMs [28, 33, 42, 44] have shown impressive code generation capabilities, they exhibit inefficiencies, such as continuing to generate unnecessary excess code snippets even after the expected code is generated. For example, Fig. 1 presents a code snippet generated by CodeLlama-7B [44], including two parts: prompt (lines 1-5) and output (lines 6-12). Generally, we provide the prompt, including method signature (lines 1) and method description (lines 3-5), as the input to CodeLlama-7B. The model generates the expected code fragment in line 6, which already meets the specified requirement. However, the model fails to terminate its inference here and continues to generate excess code (lines 7-12) until reaching the predefined maximum token generation length. The unnecessary generation of excess code not only wastes computational resources but also leads to significant energy consumption. Moreover, developers have to check the generated results and remove this excess

*Yanlin Wang is the corresponding author.

code, harming the development productivity. Therefore, *there is a clear need to understand to what extent Code LLMs are affected by this excess token generation issue and how to effectively handle it.*

In this paper, we conduct extensive preliminary studies on five Code LLMs across four popular code generation benchmarks to study to what extent Code LLMs are affected by the *excess token generation* issue and then introduce **CODEFAST**, a straightforward and effective inference acceleration approach for Code LLMs to address it.

In preliminary study, we first randomly select 100 problems from the training sets of the studied datasets (MBPP [4], MBJSP [3], MBGP [3], and MBCPP [3]). Next, we build prompts from these problems and sample generations from five representative Code LLMs, totaling 2000 generations. Then, we have three experienced programmers to conduct a human evaluation to verify and analyze the excess generation issue. The result of human evaluation shows that **excess generation is a prevalent issue in contemporary code LLMs**. To further study the implications of excess generation issue on the inference efficiency of Code LLMs, we conduct comparative experiments. We terminate the inference process of models early when detecting excess generation, which simulates an ideal scenario without excess generation. Fig. 2 shows the comparative experiment result of Code Llama series models on the MBPP dataset. By comparing the inference time under the ideal scenario and the real scenario, we can find that the excess generation issue significantly increases the inference time. This result shows that **excess token generation is a significant bottleneck that limits the inference speed of Code LLMs in code generation tasks**.

Based on the findings in our preliminary study, we propose a straightforward and effective inference acceleration approach for Code LLMs, called **CODEFAST**. The key idea of our approach is to promptly terminate the inference when the continual generation of excess tokens is detected. To achieve this, we design a core component **GenGuard**, which is a unified and lightweight gating classifier attached to a Code LLM. When generating the next token, GenGuard receives the last hidden states generated from a Code LLM and predicts whether to terminate inference at this step. To train GenGuard, we propose an automatic training framework that includes three stages: sampling, labeling, and training. In the sampling stage, the framework leverages sampling prompts and a Code LLM to generate examples with excess generation issues. In the labeling stage, the framework utilizes two code analyzers to distinguish between expected and excess code in the sampled outputs and then labels them accordingly. Finally, in the training stage, we freeze the parameters of the Code LLM and train GenGuard to predict the probability of excess tokens. This framework only necessitates users to provide sampling prompts, relieving the need and burden of manual data labeling. In the inference stage of CODEFAST, we propose a line-wise voting mechanism that can reduce misjudgments and achieve a trade-off between generation accuracy and speed. Specifically, instead of stopping the inference process of the Code LLM immediately when GenGuard predicts to stop inference for a single token, the line-wise voting mechanism collects all the predictions made by GenGuard for the whole code line. After completing a code line, it uses majority voting to decide whether to terminate the inference process.

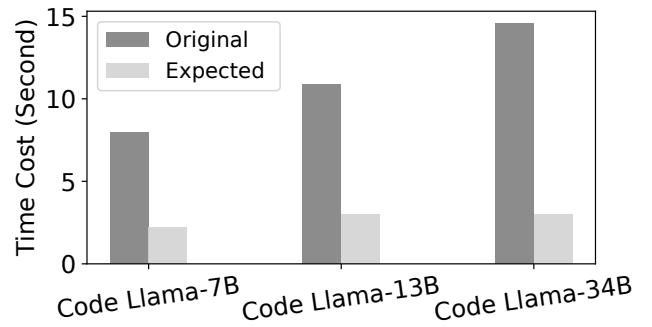


Figure 2: Experimental results of Code LLMs on the MBGP dataset: a comparison between original and expected scenarios.

To evaluate the effectiveness of CODEFAST, we conduct extensive experiments on five mainstream Code LLMs: the Code Llama series (7B, 13B, 34B) [44], StarCoder-15.5B [28], and Phind-CodeLlama [42], across four code generation datasets in different programming languages (PLs): MBPP [4], MBJSP [3], MBGP [3], and MBCPP [3]. Then, we conduct ablation studies to investigate the effectiveness of each component of our approach and explore the impact of different parameter settings on the performance of our approach. Finally, we evaluate the generalizability of our approach on three untrained code generation datasets: HumanEval [7], HumanEval-JavaScript [3], and HumanEval-Go [3]. The experimental results demonstrate that: (1) CODEFAST can significantly increase the generation speed of Code LLMs without compromising the quality of the generated code. Besides, our approach is effective across different Code LLMs and different programming languages. (2) Each component of our approach plays an important role in model performance. (3) Our approach exhibits great stability across different parameter settings. (4) Our approach exhibits great generalization capabilities on untrained datasets.

We summarize the contributions of this paper as follows:

- Though extensive experiments and human evaluation in a preliminary study, we identify a significant issue of excess token generation in Code LLMs. We find that this is a prevalent issue and has a severe impact on the efficiency of code generation. To the best of our knowledge, we are the first to investigate the excess generation issue in Code LLMs.
- We introduce GenGuard, a unified and lightweight gating classifier designed to predict the probability of stopping generation during the generation process. To obtain training data for GenGuard, we build an automated data construction framework. To make GenGuard unified across various PLs, we curate the data from datasets in multiple PLs and validate its generality by comparing it with its Mono-PL counterparts.
- We propose an effective inference acceleration method CODEFAST by utilizing GenGuard to promptly terminate the generation process. In addition, we adopt a line-voting mechanism that can reduce misjudgments and achieve a trade-off between generation accuracy and speed.

2 Background

2.1 Code Large Language Models

Code Large Language Models (Code LLMs) refer to large language models that are specifically trained for coding tasks [64, 67]. Typically, Code LLM is a Transformer-based model with a large number of parameters and has strong abilities in code generation and understanding, achieved through training on a large amount of code data [67]. For example, open-source Code LLMs such as Code Llama [44] and StarCoder [28] have also shown impressive capabilities in code generation tasks. Phind-CodeLlama [42], fine-tuned from CodeLlama-34B using high-quality code data, has surpassed the performance of GPT-4 [41] on HumanEval. Based on different Transformer architectures, current Code LLMs can be classified into two types: decoder-only Transformer models and encoder-decoder Transformer models [67]. Considering that most current Code LLMs are decoder-only Transformer models, such as Code Llama [44] and StarCoder [28], in this paper, we will focus on discussing Code LLMs with decoder-only architecture.

2.2 Inference of Code LLMs

The decoder-only Transformer architecture is initially introduced in GPT [43]. In contrast to the original Transformer [51], the decoder-only architecture is composed solely of stacked decoder layers and does not include the encoder component [43]. The core of the decoder layers is its masked attention layer. This layer employs masks to allow the model to focus only on previously generated tokens and prevents it from attending to future tokens during the decoding process [65]. This mechanism determines the autoregressive generation characteristics of the decoder-only Transformer model.

Given an input token sequence $X = \{x_1, x_2, \dots, x_n\}$, the model first maps them to context vectors $C = \{c_1, c_2, \dots, c_n\}$ in word embedding layer, where n is the length of the sequence. We use the word vectors as the initial hidden states $H^0 = \{h_1^0, h_2^0, \dots, h_n^0\} = C$, which serve as the input to the decoder layer. And then we take the hidden states output of the previous decoder layer as input to calculate the hidden states for the next decoder layer, following the Equation 1, where L represents the number of decoder layers, i represents the current index of decoder layer and M represents the mask matrix.

$$H^i = \text{DecoderLayer}(H^{i-1}, M), i \in [1, L] \quad (1)$$

After computing the hidden states of the final decoder layer, we choose the final hidden state of the last token h_n^L as the representation of the input sequence. Then, we input the representation into Language Modeling Head (LM Head) to predict the next token, following Equation 2, where W_{lm}^T is the LM Head Matrix.

$$P(x_{n+1}) = \text{Softmax}(h_n^L W_{lm}^T), \quad (2)$$

After obtaining probability vectors of next token $P(x_{n+1})$, we decode the next token x_{n+1} using greedy decoding, which means selecting the token with the highest probability following Equation 3.

$$x_{n+1} = \text{argmax}(P(x_{n+1})) \quad (3)$$

We update the output sequence $O = \{o_1\} = \{x_{n+1}\}$, and input token sequence $X = \{x_1, x_2, \dots, x_n, x_{n+1}\}$, and repeat the above inference process until the inference termination condition is reached.

Table 1: Dataset statistics.

Benchmark	Language	#Train	#Valid	#Test
MBPP	Python	374	90	500
MBJSP	JavaScript	420	46	500
MBGP	Go	396	43	500
MBCPP	C++	314	34	500
HumanEval	Python	\	\	164
HumanEval-JS	JavaScript	\	\	161
HumanEval-Go	Go	\	\	160

Here are two types of inference termination conditions: **(1) Special end token is generated.** When the special token `<EOS>` is generated, the model will stop generation. **(2) The length of the generated tokens exceeds the maximum value.** For example, in *Transformer*¹ library, there is a parameter `max_new_tokens`, when the length of the generated token sequence exceeds the value of `max_new_tokens`, the inference of the model will be terminated.

3 Experimental Design

3.1 Datasets

We use four public datasets on code generation, including MBPP [4] in Python, MBJSP [3] in JavaScript, MBGP [3] in Go, and MBCPP [3] in C++. We use the above datasets in both the preliminary study and evaluation sections. Additionally, to further validate the generalizability of our proposed approach, we utilize HumanEval [7], HumanEval-Go [3] and HumanEval-JavaScript [3]. The statistics of datasets are listed in Table 1, and the detailed description of these datasets are as follows:

- **MBPP** [4] is a frequently used benchmark in code generation, comprising 974 crowd-sourced Python programming problems. Each problem includes a task description, a solution, and three test cases.
- **HumanEval** [8] is a popular benchmark in code generation, which includes 164 hand-written Python programming problems. Every problem contains a function signature, docstring, function body, and several unit tests.
- **MBXP and Multilingual HumanEval** [3] are the multilingual versions of the MBPP and HumanEval benchmarks, sharing a similar data format. For MBXP, we use **MBJSP**, **MBGP**, and **MBCPP**, which are datasets in three popular programming languages. Because these three datasets are not pre-partitioned, we selected 500 samples to form the test set. The remaining samples are split in a 9:1 ratio to create the training and validation sets, respectively. As for Multilingual HumanEval, due to the lack of a C++ dataset, we use **HumanEval-JavaScript** and **HumanEval-Go** to verify the PL generalization ability of our approach.

3.2 Metrics and Notations

Following previous code generation studies [25, 26], we employ **Pass@k** ($k = 1, 3, 5$) as our evaluation metric to measure code generation accuracy. A requirement is considered solved if the generated program passes all test cases, and **Pass@k** is the percentage of solved problems in total problems. Additionally, we use the following metrics/notations to measure code generation quality and

¹<https://github.com/huggingface/transformers>

speed. **Length** represents the average number of tokens of generated solutions. **Time** represents the average time (in seconds) taken by a Code LLM to generate a program per sample. **Speedup** measures the improvement in code generation speed of a Code LLM, relative to its corresponding baseline performance. **ER (Excess Ratio)** quantifies the percentage of generated results that contain excess tokens. **PGWE** (percentage of generation without <EOS> Ending) indicates the percentage of results where an excess generation issue is detected and the special ending token <EOS> is not generated. This means the model will keep generating excess code until the token length reaches the maximum generation length.

3.3 Base LLMs for Code

In the preliminary study and evaluation, we select five mainstream Code LLMs as the base models for our experiments. The details of these models are as follows:

- **Code Llama Series** [44] is a series of models pre-trained on code data using Llama 2 [50], with parameter sizes ranging from 7B, 13B, to 34B. These models are currently the most powerful open-source foundation Code LLMs available. In our experiments, we select the multilingual versions of the models CodeLlama-7B, CodeLlama-13B, and CodeLlama-34B.
- **StarCoder** [28] is a 15.5B parameter model trained on 80+ programming languages, which has demonstrated remarkable performance on code generation tasks. In our experiments, we select StarCoder-15.5B as a base model.
- **Phind-CodeLlama** [42] is an advanced Code LLM fine-tuned from CodeLlama-34B on a proprietary dataset of about 80k high quality programming problems and solutions. This model outperforms GPT-4 on the HumanEval benchmark, achieving state-of-the-art performance. In our experiments, we use Phind-CodeLlama-v2 as the base model, which is the state-of-the-art version of the Phind series of models.

Although ChatGPT [39] and GPT-4 [41] have achieved notable performance in code generation tasks, we do not use them in our experiments. This is because OpenAI only provides API access for these models and does not open-source their parameters. This limitation restricts our exploration of the details of these models' inference processes.

3.4 Experimental Settings

We provide detailed experimental settings as follows.

Code generation settings. In our experiments, we deploy Code LLMs on the GPU using the bfloat16 [12] precision format. The max generated length is set to 300. We employ greedy decoding for calculating Pass@1 and nucleus sampling for calculating Pass@3 and Pass@5. Empirically, we set the temperatures following the original settings, which is 0.8 for function-level tasks [25, 26] and 0.2 for class-level tasks [11]. Notably, following previous studies [3, 25, 27, 68], our prompts for function-level code generation is designed in a partial code format, including the signature of the target function, comments with requirements, and relevant test cases. Across the experiments conducted in § 6.1, 6.2, and 6.3, we set the parameter θ_{stop} to 0.5. In experiments of § 6.4, we set a θ_{stop} of 0.7 when evaluating our approach on untrained datasets.

Training stage settings. In the sampling stage, we sample raw generations for each Code LLM mentioned in § 3.3 from the

training sets of MBPP, MJBSP, MBGP, and MBCPP. Subsequently, each Code LLM's raw generations from different programming languages are merged and labeled to construct a dataset, referred to as the Multi-PL training data. When labeling data with ChatGPT, we choose *gpt-3.5-turbo-16k* [40] to label data. In the training stage, the parameters of each Code LLM are frozen, and a linear classifier is trained to act as its GenGuard. The training process uses the AdamW [32] optimizer with a learning rate of $5e-4$ and lasts for ten epochs. Our experiments are conducted on a machine with 216 GB main memory and a Tesla A100 80GB GPU.

4 Preliminary Study

In this section, we find an efficiency issue in the inference of Code LLMs, which is the continual generation of excess tokens. This issue means that after the model generates code corresponding to a given requirement, the generation of tokens does not cease. Instead, it continues to produce undesired content that offers no assistance in solving the current problem. Considering the high inference time complexity of Code LLMs, this issue significantly increases inference time.

To gain a deeper and more comprehensive understanding of this efficiency issue in Code LLMs, we conduct extensive preliminary studies. First, we randomly select 100 problems from training sets of MBPP, MJBSP, MBGP, and MBCPP. Next, we build prompts from these problems and sample generations from the five Code LLMs described in § 3.3, totaling 2000 generations. Then, we involve three annotators, each with over five years of coding experience, and manually identify and label the excess code tokens in generations. Annotators consist of two authors and a Ph.D. student in computer science. Given prompts and generated code, annotators independently truncate the generated code to remove excess tokens without changing its functionality and annotate truncated line numbers as labels. For any inconsistent or uncertain annotations, we conduct face-to-face discussions to resolve disagreements. The agreement among annotators is verified using Krippendorff's alpha [17], and the value is 0.998, indicating a high agreement among annotators and confirming the reliability of annotated results.

- **PSRQ1:** Is the phenomenon of excess token generation a prevalent issue in contemporary code LLMs?
- **PSRQ2:** What are the implications of excess token generation on the inference efficiency of Code LLMs?

4.1 Excess Token Phenomenon (PSRQ1)

4.1.1 Qualitative Analysis. Through case studies, we find that there are three major types of issues. **Type 1: endless generation of excess functions.** The first case in Fig. 3 presents a typical example, showcasing a result generated by CodeLlama-7B on the MBPP dataset. After generating the expected function body (line 6), the generation process does not stop. The model continues to generate tokens until line 15, where it is forcibly truncated when the maximum sequence length is reached. It is notable that the extra content generated in lines 7-15 are excess functions with unhelpful content. Considering the high inference time complexity of Code LLM, such redundant generation significantly increases the inference time of models. **Type 2: undesirable test code.** The second case in Fig. 3 presents a typical example generated by Phind CodeLlama-34B

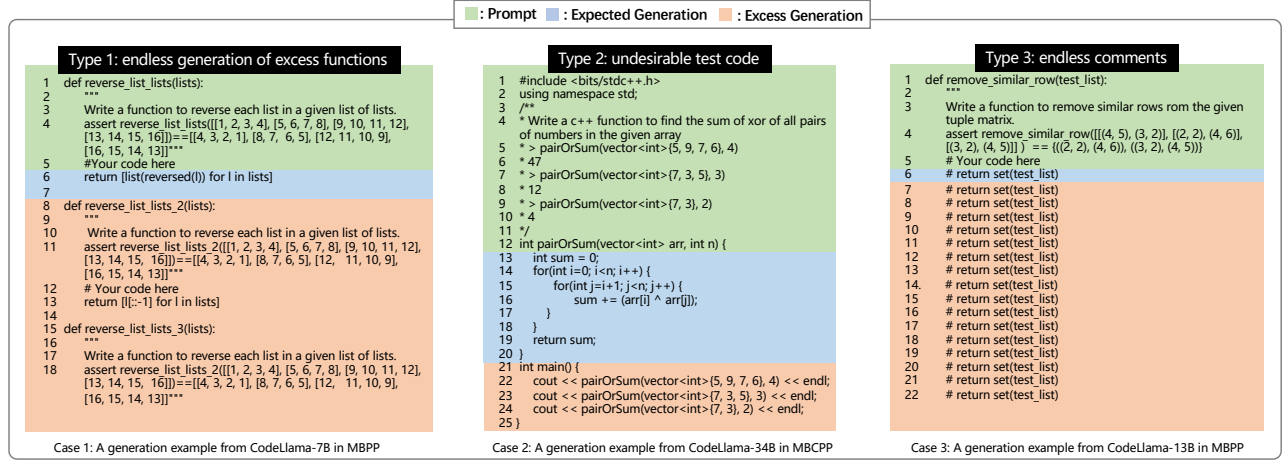


Figure 3: Results of qualitative analysis in the preliminary study.

on MBCPP. After generating the expected function (lines 12-19), the model continues to produce test code (lines 20-24). Although these test codes are somewhat related to expected generations, they are generally useless and lead to additional inference time. **Type 3: endless comments.** The third case in Fig. 3 presents a typical example generated by CodeLlama-13B on MBPP. After receiving the prompt (lines 1-5), the model generates an extensive amount of repetitive comments (lines 6-15), continuing until the maximum sequence length is reached. In this case, the model fails to generate code that solves the problem and instead produces endless comments. Ideally, we would want the model to stop inference before generating repetitive comments, thereby reducing unnecessary time consumption.

We believe that this issue mainly stems from the pre-training process of Code LLMs. During its pre-training, Code LLMs typically utilize a large dataset of code files [28, 50], which often contain multiple functions, test codes, and comments. As a result, when these models are used for function-level code generation, they tend to produce content akin to training data, leading to the generation of excess functions, test codes, etc.

4.1.2 Quantitative Analysis. To investigate to what extent this issue exists in different Code LLMs across various programming languages, we randomly select 100 prompts from MBPP, MBJSPP, MBGP, MBCPP, and sample generations from five Code LLMs as shown in § 3.3, totaling 2000 generations. Then, we manually check the outputs and mark the expected generation and excess generation for each sample. We can derive two observations from the human evaluation results shown in Table 2.

Observation 1: all Code LLMs have the excess generation issue across different programming languages. From Table 2, we find that all Code LLMs exhibit a high ER (from 59% to 100%), indicating a high frequency of excess generation. For example, outputs of CodeLlama-7B in MBPP show an ER of 100%, which means all results include excess generation. Moreover, we find that models with smaller parameter sizes, such as CodeLlama-7B, CodeLlama-13B, and StarCoder-15.5B, exhibit a higher PGWE. This means that, in some extreme cases, these models not only output excess generations but also continue generating content until reaching the maximum length of generated tokens.

Observation 2: supervised fine-tuning can mitigate the issue of excess generations in Code LLMs. Phind-CodeLlama is a model fine-tuned on CodeLlama-34B using a proprietary dataset of high-quality programming problems and solutions. From Table 2, we can see that Phind-CodeLlama, compared to CodeLlama-34B, exhibits a lower ER and shorter token length on the MBPP and MBJSPP datasets. Additionally, it also demonstrates a lower PGWE on both MBPP and MBJSPP. We believe this is due to the high-quality fine-tuning data enabling the model to better understand the intent of requirements, thereby mitigating the issue of excess generations.

PSRQ1 Summary: Overall, all five Code LLMs have severe issues of excess generations across four programming languages.

4.2 Implications on Inference Efficiency (PSRQ2)

Firstly, to explore the impact of this issue on the length of the generated token sequences, we calculate and compare the average lengths of outputs with and without excess generations. Additionally, to investigate the impact on the inference speed of Code LLMs, we prematurely terminated the model’s output generation upon the occurrence of excess generations and measured its effect on inference speed. From the experimental results shown in Table 2, we have **Observation 3: the issue of excess generations significantly increases the length of generated sequences and hence significantly increases generation time.** For instance, the average output sequence length for CodeLlama-34B on MBJSPP is 158.2 (“Original”). Ideally, if there were no excess generation, the output length should decrease to 81.8 (“Expected”). Considering the high inference time complexity of Code LLMs, these additional generated tokens will significantly increase the inference time of Code LLMs. By prematurely terminating the inference process upon the occurrence of excess generations, all Code LLMs obtain a significant increase in inference speed. For example, through early stopping, the inference speed of CodeLlama-34B on MBGP increased to 3.17 times its original speed.

PSRQ2 Summary: The issue of excess generations is a primary factor limiting the generation efficiency of Code LLMs.

Table 2: Human evaluation results of preliminary study. “Original” represents the raw output result of the model; “Expected” represents the result after manually removing excess tokens.

Model	Type	MBPP (Python)				MBJSP (JavaScript)				MBGP (Go)				MBCPP (C++)			
		Length	Time	ER	PGWE	Length	Time	ER	PGWE	Length	Time	ER	PGWE	Length	Time	ER	PGWE
CodeLlama-7B	Original	300	9.31	100%	100%	276.8	8.49	97%	81%	270.7	7.96	98%	81%	292.8	8.75	100%	96%
	Expected	51.4	1.59	0%	0%	87.6	2.65	0%	0%	72.2	2.21	0%	0%	82.5	2.56	0%	0%
CodeLlama-13B	Original	300	12.15	100%	100%	300	11.92	100%	100%	297.2	10.87	99%	97%	298.3	11.51	100%	99%
	Expected	41.8	1.65	0%	0%	83.8	3.29	0%	0%	78.7	3.03	0%	0%	79.4	3.15	0%	0%
CodeLlama-34B	Original	269.8	14.97	97%	82%	158.2	8.87	98%	3%	260.1	14.58	98%	61%	180.4	10.08	100%	14%
	Expected	52.0	2.89	0%	0%	81.8	4.60	0%	0%	81.0	4.60	0%	0%	72.8	4.12	0%	0%
StarCoder-15.5B	Original	202.3	5.29	98%	53%	231.8	6.23	100%	52%	284.6	7.69	98%	87%	257.9	7.19	100%	68%
	Expected	81.3	2.32	0%	0%	78.3	2.38	0%	0%	106.5	2.90	0%	0%	108.2	3.42	0%	0%
Phind-CodeLlama	Original	120.7	6.70	59%	4%	184.2	10.42	81%	7%	245.0	13.71	99%	45%	187.6	10.52	90%	13%
	Expected	60.7	3.39	0%	0%	93.7	5.28	0%	0%	81.3	4.62	0%	0%	74.8	4.28	0%	0%

5 Approach

In this section, we propose an effective code generation acceleration approach called **CODEFAST** as shown in Fig. 4. The key idea is to terminate the inference early when a continual generation of excess tokens is detected. CODEFAST achieves this through a key component: GenGuard, which is an additional module to predict whether to stop inference at the current step.

5.1 Data Construction Framework

Our data construction framework is shown in Fig. 4(a). By simply providing sampling prompts, our approach can automatically generate labeled data to train a specialized GenGuard for a Code LLM. This approach includes two stages, *the sampling stage* and *the labeling stage*.

5.1.1 The sampling stage. In the sampling stage, we utilize sampling prompts to collect raw generations from the model, which potentially has the issue of excess generations. The sampling prompts are in the format of partial code snippets, each including a function signature and comments that contain requirements and test cases. We collect sampling prompts from the training sets of MBPP, MBJSP, MBGP, and MBCPP. Then, we input these prompts into Code LLMs to obtain a large number of raw outputs.

5.1.2 The labeling stage. A key challenge for labeling data is how to differentiate between expected generation and excess generations in raw outputs. We design a labeling pipeline as shown in Algorithm 1. We utilize two types of code analyzers to label raw outputs from Code LLMs. **(1) Syntax-based code analyzer.** It can label the first two types of generations shown in § 4.1.1 easily by analyzing the syntax structure of generated code. This code analyzer considers all code lines related to the target function as expected generation. This includes not only the main body of the target function but also all functions that it calls. The rest of code lines are then labeled as excess generations. To achieve this, we utilize tree-sitter², an efficient parser generator that can analyze the structure of code via abstract syntax trees (ASTs). First, it builds the ASTs from the raw outputs and extracts the node of the target function using its signature. Then, it extracts all nodes called by the target function by traversing the AST. Next, it collects all extracted

function nodes and identifies the last line of these functions. Finally, the portion from the beginning up to the end of this specific line is labeled as the expected generation, and the remaining part of the output is labeled as excess generation. **(2) Semantic-based code analyzer.** It is utilized when the first analyzer fails. For instance, the third type of generation, as illustrated in Fig 3, is challenging to label by relying solely on the syntax structure of the code. It requires an understanding of the semantics of the generated code. Here, we utilize ChatGPT [39], a powerful LLM with great code understanding ability, to label the data. Inspired by the in-context-learning technique [55], we teach ChatGPT how to label the data with detailed task instructions and manually crafted demonstrations. The concrete prompt utilized in ChatGPT can be found in the Appendix of replication package [2]. Additionally, we conduct a human evaluation to verify the quality of the results truncated by ChatGPT. These annotators include two authors and one Ph.D. student in computer science. Given raw code and ChatGPT’s truncated code, they are tasked with assessing the accuracy of the truncations. Each truncation is scored on a binary scale: 1 for correct and 0 for incorrect. The final average score is 0.957, indicating a high quality of ChatGPT’s labeling. Moreover, we calculate the value of Krippendorff’s alpha [17] and obtain a result of 0.919. This indicates a high level of agreement among the annotators and confirms the reliability of the results truncated by ChatGPT.

Finally, we label each token of the expected generation portion as “continue generation” and the excess generation portion as “stop-generation”. Note that we only keep the first excess line since we expect GenGuard to stop at the earliest step.

5.2 Training of GenGuard Module

The key component of CODEFAST is the GenGuard Module, which is an additional module attached to Code LLM. The GenGuard functions as a lightweight gating classifier to control the inference process of Code LLM. To reduce extra parameters, we train a straightforward linear classifier for each Code LLM as its specialized GenGuard Module. Following previous studies [15], we freeze the parameters of Code LLM and only train the GenGuard. In this process, Code LLM is viewed as a feature extractor and encodes the code sequence into a feature vector, which represents the semantics of this code. Subsequently, the GenGuard module is trained using

²<https://github.com/tree-sitter/tree-sitter>

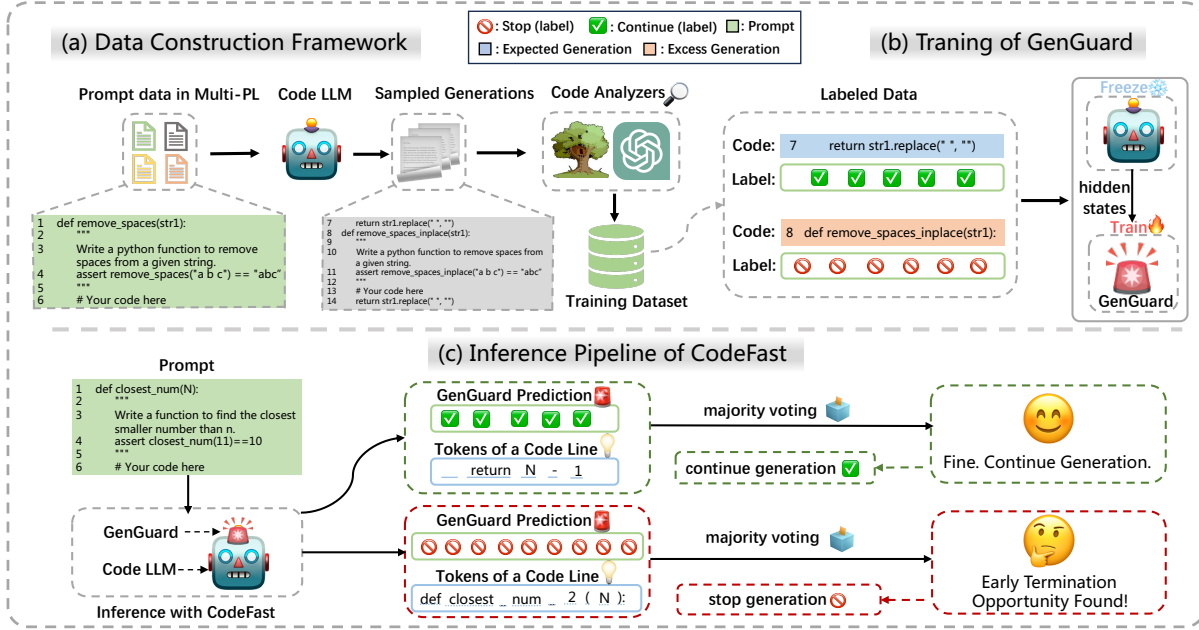


Figure 4: Overview of CODEFAST.

Algorithm 1 Labeling Algorithm

Input: Raw Output: o_{raw} , Partial Code Prompt: $prompt_{code}$, AST Builder: $ast_builder$, ChatGPT: $chatgpt$, Prompt for ChatGPT: $prompt_{gpt}$

Output: Labeled data: d_{label}

```

1:  $output\_ast \leftarrow ast\_builder(output)$ 
2:  $signature \leftarrow extract\_signature(prompt_{code})$ 
3:  $target\_func\_node \leftarrow output\_ast.locate\_node(signature)$ 
4:  $o_{expected} \leftarrow None, o_{excess} \leftarrow None$ 
5:  $last\_line\_index \leftarrow -1, node\_stack \leftarrow [target\_func\_node]$ 
6: while  $node\_stack \neq []$  do
7:   for  $node$  in  $node\_stack$  do
8:      $last\_line\_idx \leftarrow \max(last\_line\_idx, node.last\_line)$ 
9:      $called\_node\_list.append(node.find\_called\_nodes())$ 
10:    Delete  $node$  from  $node\_stack$ 
11:     $node\_stack += called\_node\_list$ 
12:   end for
13: end while
14:  $o_{expected}, o_{excess} \leftarrow split(o_{raw}, last\_line\_idx)$ 
15: if  $o_{expected} = prompt_{code}$  then
16:    $prompt \leftarrow prompt_{gpt}.replace('< raw\_output >', o_{raw})$ 
17:    $o_{expected}, o_{excess} \leftarrow chatgpt(prompt)$ 
18: end if
19: if  $o_{expected} \neq prompt_{code}$  then
20:    $o_{excess} \leftarrow extract\_first\_line(o_{excess})$ 
21:    $d_{label} \leftarrow \{ "continue" : o_{expected}, "stop" : o_{excess} \}$ 
22: else
23:    $d_{label} \leftarrow \{ "continue" : None, "stop" : None \}$ 
24: end if
25: return  $d_{label}$ 

```

these feature vectors as input. The details of GenGuard training are as follows:

Given a code sequence $X_n = (x_1, x_2, \dots, x_n)$ and label y_n at step n , we input this sequence into LLM to obtain the last hidden state h_n^L ,

following Equation 4, where L stands for the number of decoder layers in Code LLMs.

$$h_n^L = LLM(X_n) \quad (4)$$

Then, we input the last hidden state into the linear classifier to predict the probability of stopping p_{stop} at this step:

$$P_{stop} = \{p_{continue}, p_{stop}\} = Softmax(h_n^L W_{GenGuard}^T) \quad (5)$$

Finally, we use the cross entropy loss to optimize the parameters of linear classifiers, the loss is computed following 6, where y_n equals 1 to indicate “stop generation” and 0 to indicate “continue generation”.

$$Loss = -(y_n \log(p_{stop}) + (1 - y_n) \log(1 - p_{stop})) \quad (6)$$

The statistics show that the largest GenGuard module has fewer than 0.02M parameters, amounting to just 0.00005% to 0.0001% of the parameter size of our studied Code LLMs. This indicates that GenGuard is lightweight³.

5.3 Inference Pipeline of CODEFAST

In this section, we introduce the code generation process of CODEFAST. First, given an input sequence $X_n = \{x_1, x_2, \dots, x_n\}$ at step n , we input this sequence into Code LLM to predict next token x_{n+1} and last hidden states h_n^L , following Equation 7, where L represents the layer numbers of Code LLM.

$$x_{n+1}, h_n^L = LLM(X_n) \quad (7)$$

The last hidden states h_n^L at step n is a high dimensional feature vector, which includes extensive semantic information of the current input sequence X_n . Following [15], we input the feature vectors

³We present the detailed parameter information of GenGuard modules attached to five Code LLMs in Appendix-C of replication package [2].

extracted from LLM into GenGuard $W_{GenGuard}$, to predict the probability of stopping p_{stop} at the current step n :

$$P_{stop} = \{p_{continue}, p_{stop}\} = \text{Softmax}(h_n^T W_{GenGuard}^T) \quad (8)$$

Then, we compare the probability of stopping p_{stop} with hyperparameter stopping threshold θ_{stop} and terminate the inference of code generation when p_{stop} exceeds θ_{stop} .

However, there is a potential problem that may reduce the quality of the generated code. If GenGuard erroneously predicts “stop generation” when it should predict “continue generation”, the Code LLM will terminate the inference process too early, leading to incomplete code. Inspired by the concept of majority voting, we propose an algorithm named “line-voting mechanism” to mitigate this issue. The key idea of this method is to emulate the way programmers write code line by line. Instead of stopping the inference process of the Code LLM immediately when GenGuard predicts “stop generation”, the line-voting mechanism collects all the predictions made by GenGuard during the generation of a single code line. After completing the code line, it uses majority voting to decide whether to terminate the inference process. Although the line-voting mechanism may lead to the generation of more excess tokens and slightly increase inference time, it reduces the likelihood of false predictions by GenGuard, thereby enhancing the quality of the generated code.

The inference process of the CODEFAST-enhanced Code LLM is illustrated in Fig. 4(c). Upon receiving the prompt (lines 1-6), the model begins to generate tokens for the next line (line 7), while the GenGuard module simultaneously predicts whether to stop at each step. When a new line is detected, we perform majority voting on all predictions. In this case, the generation process will continue when the voting result is “continue generation” and terminate when the voting result is “stopping generation”.

6 Evaluation

We summarize the following research questions (RQs) to evaluate CODEFAST:

- **RQ1:** What is the effectiveness of CODEFAST?
- **RQ2:** How much do different components contribute?
- **RQ3:** What is the stability of CODEFAST?
- **RQ4:** What is the generalizability of CODEFAST?
- **RQ5:** Can CODEFAST be applied to other scenarios?

6.1 RQ1: Effectiveness

To answer RQ1, we evaluate the quality of code generated by Code LLMs and the generation efficiency. We evaluate the quality of generated code with Pass@k ($k=1,3,5$) and evaluate the generation efficiency with Length, Time, and Speedup as described in § 3.2. Moreover, we use five popular Code LLMs shown in § 3.3 (CodeLlama-7B, CodeLlama-13B, CodeLlama-34B, StarCoder-15.5B, and Phind-CodeLlama) across datasets in four different programming languages.

Overall Results. From the experimental results shown in Table 3, we have two observations: **(1) CODEFAST can increase the generation speed of Code LLMs significantly without compromising accuracy.** From Table 3⁴, we can find that our approach

⁴Due to space limitations, results on overall performance under Pass@3 and Pass@5 are shown in Appendix-B.1 [2]. Conclusions that hold on Pass@1 also hold for Pass@3 and Pass@5 metrics.

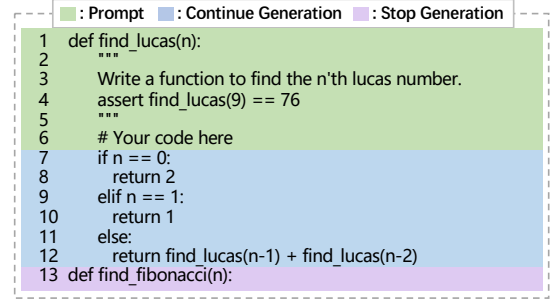


Figure 5: An example of GenGuard-enhanced Code LLM inference process.

significantly speeds up the inference speed of all Code LLMs, with the speed-up ratio ranging from 1.34 to 4.52 times. For instance, the average generation time of CodeLlama-34B decreases from 14.51 seconds to 5.89 seconds in the MBGP dataset, which means our approach brings a 2.46 times acceleration in inference. We can also find it retains the quality of the generated code from the unchanged Pass@1 score. According to preliminary studies in § 4, this is because our approach can detect the excess generations and terminate the inference process early, which retains the functionality of generated code and decreases unnecessary generations. We can find that the average token length of CodeLlama-34B decrease 263.4 to 97.6 in the MBGP dataset. Besides, we evaluate the performance of GenGuard classifiers on a manually labeled dataset⁵ introduced in Appendix-A.2 [2] with two metrics, namely precision and recall. Experimental results show that GenGuard obtains high precision and recall scores, both exceeding 0.95. Detailed information about the experiments is presented in Appendix-B.1 [2]. **(2) CODEFAST is effective in Code LLMs with different parameter sizes and different programming languages.** First, from Table 3, we can find that our approach works for all five Code LLMs with parameter sizes ranging from 7B to 34B. Second, our approach can be applied to the acceleration of code generation in different programming languages. We can find that in Table 3, our approach is effective in code generation of four programming languages such as Python, JavaScript, Go, and C++.

Case studies. Fig. 5 presents an inference example from the GenGuard-enhanced CodeLlama-7B. After inputting the prompt (lines 1-6), the GenGuard-enhanced CodeLlama-7B generate code that fulfilled requirements (lines 7-12). Following the generation of line 13, the GenGuard module predicts that generation should stop, leading to an early termination of code generation. More cases can be found in the Appendix of replication package [2].

RQ1 Summary: Overall, CODEFAST can increase the generation speed of Code LLMs significantly without compromising the quality of the generated code. Besides, our approach is effective for different Code LLMs and different programming languages.

6.2 RQ2: Ablation Study

To answer RQ2, we conduct ablation studies to investigate the impacts of the line-voting mechanism and multi-PL training data in our approach. We conduct an ablation study by removing each component at a time. Notably, when examining the impact of multi-PL

⁵Because the datasets (MBPP, MBXP, etc.) only provide test cases for evaluation. We manually construct a dataset to evaluate the prediction accuracy of GenGuard.

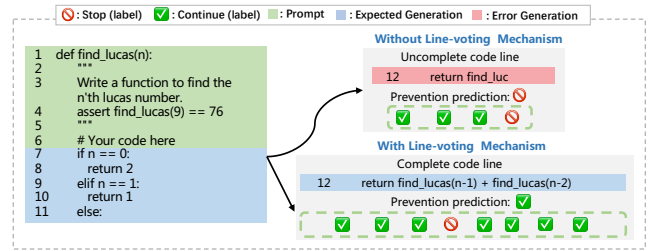
Table 3: Performance of CODEFAST on five Code LLMs across four benchmarks. P@1 is short for Pass@1. Detailed metrics descriptions are provided in § 3.2.

Model	MBPP (Python)				MBJSP (JavaScript)				MBGP (Go)				MBCPP (C++)			
	P@1	Time	Speedup	Length	P@1	Time	Speedup	Length	P@1	Time	Speedup	Length	P@1	Time	Speedup	Length
CodeLlama-7B	41.2	9.36	×1.00	299.2	47.2	8.81	×1.00	283.4	33.4	7.1	×1.00	267.0	43.0	8.44	×1.00	290.6
CodeLlama-7B _{CODEFAST}	41.0	2.07	×4.52	63.5	47.2	3.36	×2.62	110.7	33.4	3.42	×2.07	108.5	43.0	3.17	×2.66	98.8
CodeLlama-13B	44.8	11.87	×1.00	300	51.6	11.92	×1.00	300	38.4	11.94	×1.00	298.7	51.4	11.70	×1.00	299.4
CodeLlama-13B _{CODEFAST}	44.8	2.98	×3.98	71.9	51.6	4.31	×2.77	105.2	38.4	4.38	×2.73	105.3	51.4	4.00	×2.93	97.2
CodeLlama-34B	51.6	14.67	×1.00	264.8	57.4	8.37	×1.00	149.2	42.4	14.51	×1.00	263.4	55.2	10.12	×1.00	180.6
CodeLlama-34B _{CODEFAST}	51.6	3.62	×4.05	63.7	57.4	5.58	×1.50	91.5	42.4	5.89	×2.46	97.6	55.2	4.95	×2.04	79.9
StarCoder-15.5B	42.6	5.14	×1.00	204.3	32.8	6.42	×1.00	252.8	31.2	7.17	×1.00	284.6	45.2	6.15	×1.00	249.9
StarCoder-15.5B _{CODEFAST}	42.6	1.98	×2.59	73.5	32.8	1.56	×4.11	58.0	31.2	2.88	×2.49	108.4	45.2	2.55	×2.41	93.9
Phind-CodeLlama	55.8	4.89	×1.00	88.0	60.0	8.34	×1.00	147.9	42.2	14.19	×1.00	250.7	60.4	10.04	×1.00	178.6
Phind-CodeLlama _{CODEFAST}	55.8	3.65	×1.34	64.3	60.0	5.79	×1.44	96.6	42.2	5.78	×2.46	94.8	60.4	5.12	×1.96	83.0

training data, we train separate GenGuard modules for Code LLMs, each using training data from a different language. For example, in our ablation study, we train a Python GenGuard module for CodeLlama-7B using Python training data exclusively. To assess the impact of removing the line-voting mechanism, we utilize “token voting”, which means that we terminate the inference of Code LLMs immediately when GenGuard predicts to stop generation. Similar to RQ1, we conduct experiments using five Code LLMs across datasets from four programming languages.

Overall results. From Table 4⁶, we can see that: (1) **GenGuard modules trained with Multi-PL data have comparable performance to models trained with Mono-PL data.** Code LLMs enhanced with Multi-PL GenGuard achieve similar Pass@1 scores and speed acceleration as those enhanced with Mono-PL GenGuard module. This indicates that training a unified GenGuard with data from multiple PLs does not compromise its predictive ability in any of these languages. (2) **The line-voting mechanism effectively enhances the quality of code generated by GenGuard-enhanced Code LLMs.** Table 4 shows that after removing the line-voting mechanism, the speed of the GenGuard-enhanced model slightly increases, but its Pass@1 performance declines. For example, without the line-voting mechanism, the Pass@1 scores of the GenGuard-enhanced CodeLlama-13B model dropped across all four datasets. This degradation can be attributed to the GenGuard module occasionally making incorrect predictions, ending the generation process too early and producing incomplete code. In contrast, the line-voting mechanism using majority voting on the predictions of GenGuard within a line enhances the stability of the generation results.

Case studies. Fig. 6 presents an example generation with and without the line-voting mechanism. Without line-voting, the code in line 12 is incomplete. This is because the GenGuard module makes a misjudgment when generating the token “luc”, predicting that the current production process should stop. Consequently, the generation would be terminated at this point. However, when the line-voting mechanism is enabled, as the GenGuard module predicts “continue generation” for the majority of tokens in line 12, the model then completes the generation of the entire line.

**Figure 6: A comparison of GenGuard-enhanced CodeLlama-7b with & without the line-voting mechanism.**

RQ2 Summary: Overall, Multi-PL GenGuard and Mono-PL GenGuard exhibit comparable performance. And the line voting mechanism is effective and can better maintain code generation accuracy.

6.3 RQ3: Stability

In RQ3, we investigate the stability of our approach under different parameter settings. We primarily explore the impact of two parameters on the performance of CODEFAST. The first parameter is *max_new_tokens*, as mentioned in § 2.2, which represents the maximum token sequence generation length for the Code LLM. The second parameter is the *stop_threshold* θ_{stop} , indicating the probability threshold at which the GenGuard module predicts to stop generation. Due to the space limit, we only show the experiment results of the MBPP dataset in Fig. 7. Additional experimental results for MBGP, MBJSP, and MBCPP are provided in Appendix-B.3 [2].

Firstly, as illustrated in Fig. 7⁷, it is observed that the inference time for all GenGuard-enhanced Code LLMs tends to be stable with the increase of *max_new_tokens*. In contrast, the inference time for baselines significantly increases as *max_new_tokens* rises. Notably, our approach significantly decreases inference time compared to the baseline at the same *max_new_tokens* values. Furthermore, it is observed that our method maintains a Pass@1 rate comparable to baselines under different *max_new_tokens* values. This indicates that our method maintains code quality consistently as

⁶Due to space limitations, results on ablation study under Pass@3, and Pass@5 are presented in Appendix-B.2 [2]. Conclusions hold on Pass@1 also hold for Pass@3 and Pass@5 metrics.

⁷Due to space limitations, results on stability analysis under Pass@3 and Pass@5 are presented in Appendix-B.3 [2]. Conclusions hold on Pass@1 also hold for Pass@3 and Pass@5 metrics.

Table 4: Ablation study results.

Model	Type	MBPP(Python)		MBJSP(Javascript)		MBGP(Go)		MBCPP(C++)	
		Pass@1	Speedup	Pass@1	Speedup	Pass@1	Speedup	Pass@1	Speedup
CodeLlama-7B	CODEFAST	41.0	$\times 4.52$	47.2	$\times 2.62$	33.4	$\times 2.07$	43.0	$\times 2.66$
	w/o MultiPL	41.2	$\times 4.67$	47.2	$\times 2.60$	32.6	$\times 2.16$	43.0	$\times 2.72$
	w/o LineVoting	41.0	$\times 5.81$	46.2	$\times 3.49$	33.4	$\times 2.29$	42.8	$\times 2.78$
CodeLlama-13B	CODEFAST	44.8	$\times 3.98$	51.6	$\times 2.77$	38.4	$\times 2.73$	51.4	$\times 2.93$
	w/o MultiPL	44.8	$\times 4.03$	51.6	$\times 2.76$	38.4	$\times 2.71$	51.4	$\times 2.93$
	w/o LineVoting	42.8	$\times 5.41$	49.4	$\times 3.83$	38.0	$\times 3.09$	49.8	$\times 3.16$
CodeLlama-34B	CODEFAST	51.6	$\times 4.05$	57.4	$\times 1.50$	42.4	$\times 2.46$	55.2	$\times 2.04$
	w/o MultiPL	51.4	$\times 4.08$	57.4	$\times 1.49$	42.4	$\times 2.46$	55.2	$\times 2.02$
	w/o LineVoting	50.8	$\times 5.13$	56.4	$\times 2.04$	42.2	$\times 2.69$	55.2	$\times 2.21$
StarCoder-15.5B	CODEFAST	42.6	$\times 2.59$	32.8	$\times 4.11$	31.2	$\times 2.49$	45.2	$\times 2.41$
	w/o MultiPL	42.6	$\times 2.59$	32.8	$\times 4.11$	31.2	$\times 2.09$	45.2	$\times 2.41$
	w/o LineVoting	39.2	$\times 6.21$	32.4	$\times 4.89$	30.8	$\times 2.87$	45.0	$\times 2.68$
Phind-CodeLlama	CODEFAST	55.8	$\times 1.34$	60.0	$\times 1.44$	42.2	$\times 2.46$	60.4	$\times 1.96$
	w/o MultiPL	55.8	$\times 1.33$	60.0	$\times 1.41$	42.2	$\times 2.47$	60.4	$\times 1.94$
	w/o LineVoting	53.4	$\times 1.72$	58.8	$\times 1.92$	41.6	$\times 2.94$	59.2	$\times 2.16$

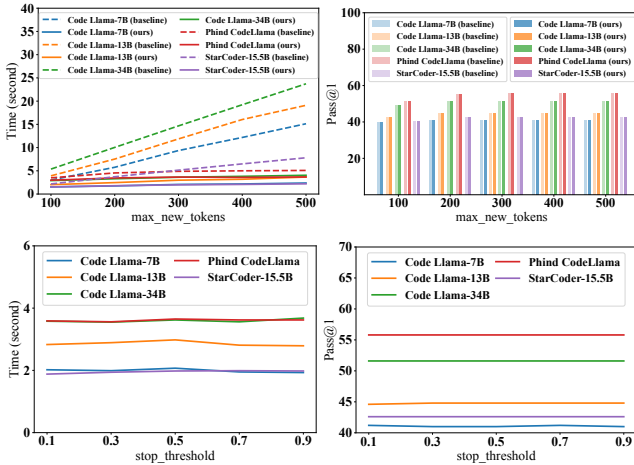


Figure 7: Stability analysis of CODEFAST in MBPP.

max_new_tokens varies. Secondly, our approach demonstrates stable performance in both inference speed and Pass@1 across various stop threshold values θ_{stop} . This is because GenGuard is highly confident in stopping prediction and can tolerate a wide range of θ_{stop} values. We analyze the stopping prediction probability distribution of GenGuard. The results indicate that the stopping probability is distributed between 0.9 and 1 when predicting true excess tokens and between 0 and 0.1 when predicting true expected tokens. The experimental results are in Appendix-B.3 [2].

RQ3 Summary: Overall, CODEFAST exhibits great stability across different parameter settings.

6.4 RQ4: Generalizability

To further validate the generalizability of CODEFAST, we evaluate the performance of five GenGuard-enhanced models using three

Table 5: Performance of CODEFAST in untrained datasets. Python, JavaScript, and Go are short for HumanEval, HumanEval-JavaScript, and HumanEval-Go, respectively.

Model	Python		JavaScript		Go	
	P@1	Speedup	P@1	Speedup	P@1	Speedup
CodeLlama-7B	34.1	$\times 1.00$	32.9	$\times 1.00$	15.0	$\times 1.00$
CodeLlama-7B_{CODEFAST}	34.1	$\times 2.71$	32.9	$\times 1.99$	15.0	$\times 2.11$
CodeLlama-13B	35.4	$\times 1.00$	39.8	$\times 1.00$	20.0	$\times 1.00$
CodeLlama-13B_{CODEFAST}	35.4	$\times 2.75$	39.8	$\times 2.48$	20.0	$\times 2.53$
CodeLlama-34B	48.8	$\times 1.00$	44.1	$\times 1.00$	22.5	$\times 1.00$
CodeLlama-34B_{CODEFAST}	48.6	$\times 2.43$	44.1	$\times 1.38$	22.5	$\times 2.17$
StarCoder-15.5B	33.5	$\times 1.00$	15.5	$\times 1.00$	15.0	$\times 1.00$
StarCoder-15.5B_{CODEFAST}	33.5	$\times 1.18$	15.5	$\times 5.01$	15.0	$\times 2.21$
Phind-CodeLlama	70.7	$\times 1.00$	68.9	$\times 1.00$	34.3	$\times 1.00$
Phind-CodeLlama_{CODEFAST}	70.3	$\times 1.48$	68.9	$\times 1.19$	34.3	$\times 1.87$

datasets in different PLs (Python, JavaScript, Go) from the multilingual HumanEval benchmark. Our models are evaluated on the test sets of these datasets without any further training. The experimental results are shown in Table 5⁸. More detailed results with time consumption and average lengths can be found in Appendix-B.4 of replication package [2]. We can observe that after being enhanced by the GenGuard module, the generation speed of all five Code LLMs increases across three programming languages. Notably, this improvement is achieved while maintaining unchanged Pass@1 scores, indicating that CODEFAST exhibits strong generalization capabilities, achieving robust performance on untrained datasets.

RQ4 Summary: Overall, CODEFAST has robust performance on untrained datasets, exhibiting great generalization capabilities.

⁸Due to space limitations, results on generalizability experiments under Pass@3 and Pass@5 are presented in Appendix-B.4 [2]. Conclusions hold on Pass@1 also hold for Pass@3 and Pass@5 metrics.

6.5 RQ5: Application to other scenarios

To explore whether CODEFAST can be applied to other scenarios, we evaluate it on a class-level code generation dataset named ClassEval [11]. It includes 100 class-level Python code generation tasks. Each task takes a class skeleton as input, which includes both class-level information (import statements, class name, etc.) and method-level information (method signature, functional description, etc.) [11]. The model is asked to complete this class skeleton by infilling the implementation of unfinished methods. In our experiments, we utilize method-by-method generation strategies, which are recommended by ClassEval to evaluate models that are not GPT-4 and GPT-3.5 [11]. These strategies mean that the model generates the class method-by-method until the whole class is completed. These strategies include incremental generation strategy and compositional generation strategy [11]. We utilize both of them for evaluation and find issues of excess generation when generating methods for the given class. Detailed information about ClassEval and generation strategies can be found in Appendix-A.1 [2].

Overall results. The experimental results with the incremental generation strategy are presented in Table 6. We can see that CODEFAST effectively increases the generation speed for all five Code LLMs on ClassEval, while the performance does not change across the three metrics (Pass@1, Pass@3 and Pass@5). Due to space limitations, we present experimental results with the compositional generation strategy in Appendix-B.5 [2], and results also demonstrate the efficiency of CODEFAST.

Table 6: Performance of CODEFAST on ClassEval with incremental generation strategy. P@k represents the probability of top k-generated classes that successfully pass unit tests.

Model	P@1	P@3	P@5	Time	Speedup
CodeLlama-7B	16.0	18.4	20.0	39.5	×1.00
CodeLlama-7B_{CODEFAST}	16.0	18.4	20.0	18.9	×2.09
CodeLlama-13B	16.0	21.6	24.0	50.0	×1.00
CodeLlama-13B_{CODEFAST}	16.0	21.6	24.0	19.1	×2.62
CodeLlama-34B	26.0	28.7	31.0	70.8	×1.00
CodeLlama-34B_{CODEFAST}	26.0	28.7	31.0	33.2	×2.13
StarCoder-15.5B	18.0	20.8	22.0	38.6	×1.00
StarCoder-15.5B_{CODEFAST}	18.0	20.8	22.0	22.2	×1.74
Phind-CodeLlama	18.0	24.9	26.0	57.1	×1.00
Phind-CodeLlama_{CODEFAST}	18.0	24.9	26.0	34.6	×1.65

RQ5 Summary: Our approach can be applied to class-level code generation scenarios and effectively increases generation speed.

7 Related Work

7.1 LLM-based Code Generation

Code generation has been extensively studied in recent years [16, 23, 30, 52] and currently the paradigm has been shifted to LLM-based code generation [14, 19, 25, 26, 31, 33, 53, 56, 63, 66]. Some works [19, 33, 42, 56] boost the code generation ability through Supervised Fine-Tuning (SFT). For instance, Phind-CodeLlama [42] achieves superior performance over GPT-4 through SFT on high-quality code datasets based on the CodeLlama-34B model. Besides, some works [25, 26, 31, 54, 63] propose effective prompt techniques

to enhance the code generation ability of Code LLMs. For instance, Li et al. [27] propose AceCoder, which retrieves programs related to given requirements to create a prompt, enabling the model to learn from these examples and generate high-quality code. Other works [35, 68] propose decoding strategies for Code LLMs to improve the performance of code generation. For example, Zhu et al. [68] propose AdapT sampling to improve the performance of code generation through adaptive adjusting the decoding temperature.

7.2 Efficient Inference of LLMs

Recently, many studies attempt to improve the inference efficiency of LLMs. Some works [9, 10, 21] increase the inference speed of LLMs by optimizing memory management and data access. Dao et al. [9, 10] propose FlashAttention to accelerate inference by reorganizing attention computation through a tiling approach, which significantly decreases GPU memory read/write operations. Other works [24, 61] accelerate the inference of LLMs by decreasing the computational time of predicting each token. Leviathan et al. [24] propose speculative decoding, which employs a small model to predict each token and a large model to verify it. This approach not only enhances the generation speed but also ensures the quality of the outputs. Besides, many works attempt to increase the inference efficiency of LLMs in code intelligence tasks [13, 47–49]. For example, Sun et al. [48] propose an efficient inference approach for Code LLMs. This method utilizes a Transformer-based estimator to assess the quality of prompts and prevent the completion of low-quality prompts in advance. Compared with these works, CODEFAST improves inference efficiency for Code LLMs by preventing the generation of excess tokens. Our approach is complementary to existing efficient inference approaches, allowing for the synergy to further enhance inference efficiency for Code LLMs. However, these current technologies, while effective at enhancing the inference speed of large language models, have not been tailored specifically to the unique characteristics of code generation tasks.

8 Discussion

Comparison with relevant techniques. Recently, Sun et al. [47] propose an effective and similar method named SEC to accelerate code completion speed by training a classifier to prevent *erroneous generation*. Moreover, some brute-force tricks specific for Python such as terminating generation upon “\n\ndef” may easily remove excess generation.

Compared with SEC, our approach exhibits several key differences. First, the SEC is trained to classify *erroneous generation* rather than classify *excess generation*. This difference stems from the fact that the training code used by SEC does not include *excess generation*, and the labeling method can not label “stop” at excess token, preventing SEC from learning to eliminate *excess generation*. The experiment results⁹ in Appendix-D.1 [2] show that our approach can increase generation speed while unchanging the model performance. Second, the classifier of CODEFAST is designed and evaluated for multiple PLs, while the SEC classifier is intended for mono-PL. Third, CODEFAST proposes an effective line-voting mechanism, which can effectively increase the accuracy of generated code as shown in experiments of RQ2. Fourth, CODEFAST offers

⁹We compare SEC and CODEFAST using the MBPP dataset.

a more lightweight training process while SEC requires training extra intermediate LM head for each Transformer layer.

Compared with brute-force tricks, our approach demonstrates several distinct advantages. First, experimental results in Appendix-D.2 [2] demonstrate that CODEFAST can significantly increase the generation speed. Second, our method can be applicable to multiple programming languages, while these brute-force tricks are for a specific programming language (Python). Third, CODEFAST introduces an automatic training method, without the need for manually collecting specific patterns such as “\ndef”. Detailed discussion and experiments about relevant techniques are presented in Appendix-D [2].

Application scope and limitations. CODEFAST aims to accelerate code generation by preventing the excess generation. Our study primarily focuses on function-level and class-level code generation, where we have conducted extensive experiments and demonstrated the efficiency of CODEFAST. In the future, we will investigate the excess generation issue in more complicated code generation scenarios and evaluate CODEFAST more comprehensively in other scenarios such as file-level and repository-level code generation.

9 Conclusion

In this paper, we propose a straightforward and effective approach CODEFAST for accelerating the code generation speed of Code LLMs and show its effectiveness through extensive experiments. In our preliminary studies, we present the issue of excess generations and show that this issue is a significant factor limiting code generation speed. Then, we propose an effective inference acceleration approach CODEFAST, which terminates the inference process early when detecting excess generations with GenGuard module. Additionally, we propose a data construction framework to obtain training data for the GenGuard automatically. Experimental results show that CODEFAST effectively increases the code generation speed of Code LLMs without compromising the quality of generated code. We believe CODEFAST can effectively improve the efficiency of Code LLMs in real-world scenarios and be utilized in IDE in the future. Our code and data are available at <https://github.com/DeepSoftwareAnalytics/CodeFast>.

Acknowledgments

The work is supported by CCF-Huawei Populus Grove Fund CCFHuaweiSE202301 and the National Key Research and Development Program of China (2023YFB2704801).

References

- [1] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K Lahiri, and Sri-ram K Rajamani. 2023. Guiding Language Models of Code with Global Context using Monitors. *arXiv preprint arXiv:2306.10763* (2023).
- [2] Deep Software Analytics. 2024. CodeFast. <https://github.com/DeepSoftwareAnalytics/CodeFast>.
- [3] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual Evaluation of Code Generation Models. In *The Eleventh International Conference on Learning Representations*.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [5] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B Ashok, and Shashank Shet. 2024. CodePlan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 675–698.
- [6] Angelica Chen, Jérémy Scheurer, Tomasz Korbak, Jon Ander Campos, Jun Shern Chan, Samuel R Bowman, Kyunghyun Cho, and Ethan Perez. 2023. Improving code generation by training with natural language feedback. *arXiv preprint arXiv:2303.16749* (2023).
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. *Advances in neural information processing systems* 31 (2018).
- [9] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. (2023).
- [10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- [11] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).
- [12] Google Cloud. 2020. bfloat16: The secret to high performance on Cloud TPUs. <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>. Accessed: 2023-04-04.
- [13] Wenchao Gu, Yanlin Wang, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Michael Lyu. 2022. Accelerating Code Search with Deep Hashing and Code Classification. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2534–2544.
- [14] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.
- [15] Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2024. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in neural information processing systems* 36 (2024).
- [16] Shirley Anugrah Hayati, Raphael Olivier, Pravalika Avvaru, Pengcheng Yin, Anthony Tomasic, and Graham Neubig. 2018. Retrieval-based neural code generation. In *EMNLP*.
- [17] Andrew F Hayes and Klaus Krippendorff. 2007. Answering the call for a standard reliability measure for coding data. *Communication methods and measures* 1, 1 (2007), 77–89.
- [18] Baizhou Huang, Shuai Lu, Weizhu Chen, Xiaojun Wan, and Nan Duan. 2023. Enhancing Large Language Models in Coding Through Multi-Perspective Self-Consistency. *arXiv preprint arXiv:2309.17272* (2023).
- [19] Naman Jain, Tianjun Zhang, Wei-Lin Chiang, Joseph E Gonzalez, Koushik Sen, and Ion Stoica. 2023. LLM-Assisted Code Cleaning For Training Accurate Code Generators. *arXiv preprint arXiv:2311.14904* (2023).
- [20] Hui Jiang, Chulun Zhou, Fandong Meng, Biao Zhang, Jie Zhou, Degen Huang, Qingqiang Wu, and Jinsong Su. 2021. Exploring dynamic selection of branch expansion orders for code generation. *arXiv preprint arXiv:2106.00261* (2021).
- [21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [22] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.
- [23] Triet HM Le, Hao Chen, and Muhammad Ali Babar. 2020. Deep learning for source code modeling and generation: Models, applications, and challenges. *ACM Computing Surveys (CSUR)* 53, 3 (2020), 1–38.
- [24] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [25] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Structured Chain-of-Thought Prompting for Code Generation. *arXiv preprint arXiv:2305.06599* (2023).
- [26] Jia Li, Ge Li, Chongyang Tao, Huangzhao Zhang, Fang Liu, and Zhi Jin. 2023. Large Language Model-Aware In-Context Learning for Code Generation. *arXiv preprint arXiv:2310.09748* (2023).
- [27] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. AceCoder: An Effective Prompting Technique Specialized in Code Generation. *ACM Transactions on Software Engineering and Methodology* (2024).
- [28] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [29] Xin-Ye Li, Jiang-Tian Xue, Zheng Xie, and Ming Li. 2023. Think Outside the Code: Brainstorming Boosts Large Language Models in Code Generation. *arXiv*

- preprint arXiv:2305.10679* (2023).
- [30] Wang Ling, Edward Grefenstette, Karl Moritz Hermann, Tomáš Kočíský, Andrew Senior, Fumin Wang, and Phil Blunsom. 2016. Latent predictor networks for code generation. In *ACL*.
 - [31] Yue Liu, Thanh Le-Cong, Ratnadira Widayarsi, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D Le, and David Lo. 2023. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *arXiv preprint arXiv:2307.12596* (2023).
 - [32] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
 - [33] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv preprint arXiv:2306.08568* (2023).
 - [34] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2023. Octopack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124* (2023).
 - [35] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*. PMLR, 26106–26128.
 - [36] Lunyiu Nie, Jiuding Sun, Yanlin Wang, Lun Du, Shi Han, Dongmei Zhang, Lei Hou, Juanzi Li, and Jidong Zhai. 2023. Unveiling the black box of LLMs with semantic anchors: towards interpretable neural semantic parsing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13400–13408.
 - [37] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2023. Demystifying GPT Self-Repair for Code Generation. *arXiv preprint arXiv:2306.09896* (2023).
 - [38] OpenAI. 2021. OpenAI Code. <https://openai.com/blog/openai-code>.
 - [39] OpenAI. 2022. ChatGPT. <https://openai.com/blog/chatgpt>.
 - [40] OpenAI. 2023. gpt-3.5-turbo. <https://platform.openai.com/docs/models/gpt-3-5-turbo>.
 - [41] OpenAI. 2023. GPT-4. <https://openai.com/research/gpt-4>.
 - [42] Phind. 2023. Phind-CodeLlama-34B-v2. <https://huggingface.co/Phind/Phind-CodeLlama-34B-v2>.
 - [43] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
 - [44] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
 - [45] Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards Efficient Fine-tuning of Pre-trained Code Models: An Experimental Study and Beyond. *arXiv preprint arXiv:2304.05216* (2023).
 - [46] Ensheng Shi, Fengji Zhang, Yanlin Wang, Bei Chen, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. SoTaNa: The Open-Source Software Development Assistant. *arXiv preprint arXiv:2308.13416* (2023).
 - [47] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, and Li Li. 2024. When Neural Code Completion Models Size up the Situation: Attaining Cheaper and Faster Completion through Dynamic Model Inference. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE, 906–917.
 - [48] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. 2023. Don't Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 324–325.
 - [49] Zhensu Sun, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2024. AI Coders Are Among Us: Rethinking Programming Language Grammar Towards Efficient Code Generation. In *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*.
 - [50] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [52] Yanlin Wang and Hui Li. 2021. Code completion by modeling flattened abstract syntax trees as graphs. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 14015–14023.
 - [53] Yanlin Wang, Yanli Wang, Daya Guo, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. RLCoder: Reinforcement Learning for Repository-Level Code Completion. (2024).
 - [54] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Source code is all you need. In *International Conference on Machine Learning*.
 - [55] Ruixuan Xiao, Yiwen Dong, Junbo Zhao, Runze Wu, Minmin Lin, Gang Chen, and Haobo Wang. 2023. FreeAL: Towards Human-Free Active Learning in the Era of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14520–14535.
 - [56] Prateek Yadav, Qing Sun, Hantian Ding, Xiaopeng Li, Dejiao Zhang, Ming Tan, Xiaofei Ma, Parminder Bhatia, Ramesh Nallapati, Murali Krishna Ramanathan, et al. 2023. Exploring continual learning for code generation models. *arXiv preprint arXiv:2307.02435* (2023).
 - [57] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pretrained Models. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE, 428–439.
 - [58] Daoguang Zan, Bei Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-library-oriented code generation with large language models. *arXiv preprint arXiv:2307.15370* (2023).
 - [59] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2023. Large language models meet NL2Code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7443–7464.
 - [60] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484.
 - [61] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. *arXiv preprint arXiv:2309.08168* (2023).
 - [62] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 769–787.
 - [63] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. 2023. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510* (2023).
 - [64] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2023. A Survey on Language Models for Code. *arXiv preprint arXiv:2311.07989* (2023).
 - [65] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
 - [66] Dewu Zheng, Yanlin Wang, Ensheng Shi, Ruikai Zhang, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. 2024. Towards more realistic evaluation of LLM-based code generation: an experimental study and beyond. *arXiv preprint arXiv:2406.06918* (2024).
 - [67] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2023. A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends. *arXiv preprint arXiv:2311.10372* (2023).
 - [68] Yuqi Zhu, Jia Allen Li, Ge Li, Yunfei Zhao, Jia Li, Zhi Jin, and Hong Mei. 2023. Improving Code Generation by Dynamic Temperature Sampling. *arXiv preprint arXiv:2309.02772* (2023).